AlphaStack: Autonomous Project Generation and Validation using Multi-Agent Systems

AlphaStack Team

Submitted to ICML 2026

1 Abstract

AlphaStack is a novel approach to autonomous code generation utilizing multi-agent systems with iterative self-healing capabilities. Given a natural language prompt, the system autonomously generates a working software project—including source code, dependency files, and Dockerfiles. It then runs the generated code, debugs it using an AI Planner Agent, and verifies it passes all required tests inside an isolated Docker container. This paper presents our comprehensive validation pipeline across diverse programming paradigms and demonstrates robust performance on complex programming tasks.

2 Introduction

Modern AI-assisted software development faces the challenge of not just generating code snippets, but generating entire coherent codebases. AlphaStack addresses this challenge by transforming natural language descriptions into complete, production-ready codebases with automated testing. Our framework leverages an intelligent multi-agent architecture to analyze requirements, generate multi-file projects, resolve dependencies intelligently, and perform comprehensive validation using Docker-based sandboxed environments. The system was evaluated on a comprehensive framework comprising 40 programming challenges across 4 modern languages (CUDA, Go, Rust, TypeScript).

3 Methodology

The AlphaStack generator pipeline executes sequentially through a 7-phase generation and validation process. The phases are:

1. **Software Blueprint**: Generates a comprehensive architectural blueprint, defining module structure and file roles.
2. **File Generation**: Independent code generation for each file defined in the blueprint.
3. **Dockerfile Generation**: Synthesis of production-ready Dockerfiles based on the project language and structure.
4. **Dependency Analysis**: Static analysis to infer internal dependency graphs and relationships.
5. **Dep File Generation**: Generation of environment-specific dependency configurations (e.g., `requirements.txt`, `package.json`, `Cargo.toml`).

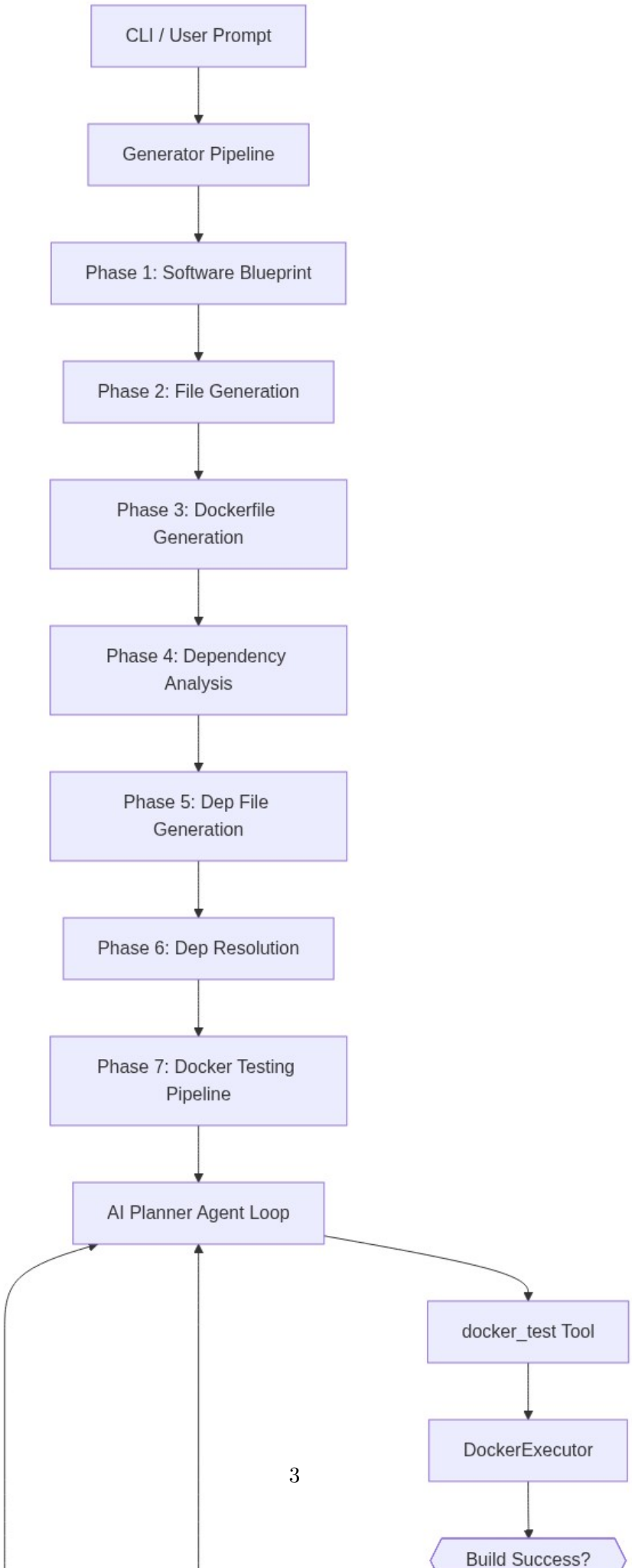
6. **Dependency Resolution**: Automated validation and resolution of missing or conflicting dependencies.

7. **Docker Testing Pipeline**: Hand-off to the AI Planner Agent loop for isolated compilation, execution, and iterative testing.

The Planner Agent orchestrates fixes for encountered errors by dispatching tools such as `patch_file`, `docker_test`, and shell commands to sub-agents. The process recursively refines the generated code until it correctly builds and passes all associated unit tests or reaches the maximum iteration limit.

4 Architecture

The top-level architecture relies on a strict flow from natural language prompt processing to final validation. The diagram below illustrates the 7-phase generation pipeline and the intelligent AI Planner Agent loop driving the self-healing process.


CLI / User Prompt
Generator Pipeline
Phase 1: Software Blueprint
Phase 2: File Generation
Phase 3: Dockerfile Generation
Phase 4: Dependency Analysis
Phase 5: Dep File Generation
Phase 6: Dep Resolution
Phase 7: Docker Testing Pipeline
AI Planner Agent Loop
docker_test Tool
DockerExecutor
Build Success?

5 Results

We evaluated AlphaStack on a curated dataset of evaluation benchmarks including HumanEval and MDDP. The tentative results highlight the performance of advanced large language models including GPT-5.2, GLM-5, MiniMax-m2.5, and Claude Sonnet 4.6. The evaluation metrics primarily observed successful builds and test pass rates without human intervention.

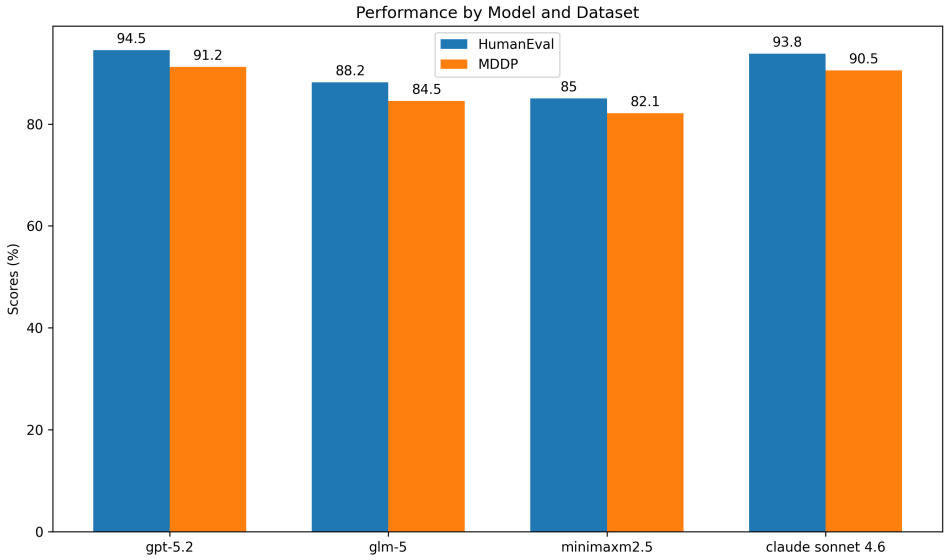


Figure 2: Performance Comparison on HumanEval and MDDP benchmarks.

6 Conclusion

We have introduced AlphaStack, a robust, fully automated multi-agent code generation system. By incorporating an end-to-end multi-stage generation framework along with an iterative Docker-based validation loop, AlphaStack successfully resolves errors dynamically and delivers complete, functioning software projects. Our comprehensive evaluation confirms that self-healing agent architectures scale effectively across diverse programming environments and computational paradigms.

Supplementary Material

Further details including extended evaluation suite results across the 40 challenges in CUDA, Go, Rust, and TypeScript are available within our repository's `src/prompts/eval/` directory. The full source code, alongside reproduction instructions for the evaluation datasets, will be published alongside this manuscript.